REST API и gRPC
Балансировка и кэширование

Семинар 5

Кирилл Дикалин

Институт прикладных компьютерных наук
Университет ИТМО

План занятия

Вопрос	Проверка
Как устроен вызов сервиса?	HTTP, gRPC, 304, дедлайн
Кто получит следующий запрос?	Две реплики и один канал
Можно ли ответить из памяти?	TTL, инвалидация, stampede
Как собрать решение?	Архитектурный разбор

Один каталог, три инженерных решения

Карточка товара **book-1**. Браузер показывает цену, внутренний сервис читает ту же карточку.

1. **Контракт:** какие данные и ошибки понимают обе стороны?
2. **Маршрут:** какая реплика выполняет работу?
3. **Повторное использование:** насколько старый ответ ещё допустим?

> Каталог может показывать недавнюю цену. При оформлении заказа цену подтверждает источник истины.

Как Web и RPC решали разные задачи

Когда	Идея	Какую проблему решает
1984	Birrell и Nelson: реализация RPC	Вызов процедуры между машинами
2000	Roy Fielding формализует REST	Независимая эволюция компонентов Web
2015	Google публикует gRPC	Открытый RPC поверх HTTP/2, опыт Stubby
2016	gRPC 1.0	Стабильный протокол и API проекта

Web: общий язык взаимодействия с ресурсами.
RPC: явный контракт операций конкретного сервиса.

REST и единообразный интерфейс

REST задаёт ограничения архитектуры: **клиент–сервер, stateless, кэшируемость, единый интерфейс, слои**. Code-on-demand необязателен.

Единый интерфейс включает ресурсы и представления, самодостаточные сообщения и **гипермедиа как управление переходами**.

```
GET /items/book-1 HTTP/1.1
Accept: application/json
```

Наличие JSON и адресов ресурсов само по себе не доказывает соответствие всем ограничениям REST.

HTTP: семантика важнее формы URL

Операция	Значение	Повтор
GET /items/book-1	Получить представление	Safe и idempotent
PUT /items/book-1	Установить заданное состояние	Idempotent
POST /orders	Создать заказ по контракту API	Сам по себе не idempotent
DELETE /items/book-1	Удалить ресурс	Idempotent, ответы могут различаться

200 / 404 / 409 / 503 сообщают разные исходы.
Идемпотентность описывает эффект повторения, а не одинаковые байты ответа.

Условный запрос и HTTP-кэш

```
HTTP/1.1 200 OK
ETag: "v1"
Cache-Control: public, max-age=10
```

```
GET /items/book-1
If-None-Match: "v1"

HTTP/1.1 304 Not Modified
```

Fresh cache hit: ответ без обращения к origin.
304: проверка состоялась, тело повторно не передали.

no-cache: проверять перед повторным использованием. no-store: не сохранять. private: не хранить в shared cache. Vary добавляет измерения выбора представления.

gRPC: контракт и инфраструктура вызова

```
service CatalogService {
  rpc GetItem(GetItemRequest) returns (GetItemResponse);
}
message GetItemRequest {
  string id = 1;
}
```

В этом фрагменте контракта запрос содержит идентификатор товара.

Из контракта генерируются сообщения и клиентский stub.
gRPC обычно использует **Protobuf + HTTP/2**, передаёт metadata и статус вызова.

NOT_FOUND отличается от **UNAVAILABLE** и **DEADLINE_EXCEEDED**.

Результат RPC определяется полем **grpc-status**: **0 — успех**, остальные коды — ошибки. HTTP 200 не гарантирует успешный RPC.

Protobuf и эволюция контракта

```
message Item {
  string id = 1;
  string title = 2;
  int64 price_rub = 3;
  uint64 version = 4;
  // После удаления поля его номер нельзя переиспользовать.
}
```

- Номер поля участвует в wire format. Имя недостаточно.
- Удалённые номера и имена резервируем через **reserved**.
- Добавление поля часто совместимо на wire-уровне, но бизнес-смысл тоже проверяем.
- Отсутствие scalar-поля и значение по умолчанию различимы не всегда. Для presence нужна соответствующая схема, например **optional**.

Дедлайн и поток сообщений

Вид RPC	Обмен
Unary	Один запрос, один ответ
Server streaming	Один запрос, поток ответов
Client streaming	Поток запросов, один ответ
Bidirectional	Два независимых потока

Дедлайн ограничивает ожидание клиента. Серверный код должен реагировать на отмену.

HTTP/2 мультиплексирует потоки в соединении. Потеря TCP-пакета всё ещё может задержать доставку данных нескольких потоков.

Выбор интерфейса по условиям задачи

Условия	Что оценивать
Браузер, партнёры, публичный API	HTTP-семантика, инструменты, документация, shared cache
Внутренние типизированные сервисы	Генерация клиентов, gRPC-статусы, deadline, streaming
Большая доля сетевых/CPU-затрат	Размер сообщений, кодирование, reuse соединений
Долгие операции в БД	Сначала длительность работы и очереди

Сравниваем одинаковую бизнес-операцию, нагрузку и SLO. Время одного запроса на ноутбуке не выбирает архитектуру.

HTTP JSON тоже работает поверх HTTP/2. gRPC-Web требует отдельной проверки браузерного пути.

Демо A. Один товар, два интерфейса

Прогноз: какие части ответа совпадут, где увидим ошибку?

1. Получить **book-1** по HTTP и gRPC.
2. Сравнить бизнес-поля и служебную информацию.
3. Повторить HTTP-запрос с **If-None-Match**.
4. Запросить неизвестный товар и исчерпать дедлайн RPC.

```
.venv/bin/python tools/demo.py contracts --step conditional
```

Наблюдаем: одинаковую карточку, 304 без тела, NOT_FOUND, DEADLINE_EXCEEDED.

Что доказал первый эксперимент

Наблюдение	Вывод
Одинаковые id, цена и версия	У book-1 совпали три поля через HTTP и gRPC
304, пустое тело; origin_reads = 2	Тело не передали; источник прочитали снова
NOT_FOUND	Ошибка предметной области читается через RPC-статус
DEADLINE_EXCEEDED	Клиент ограничил время ожидания

Не измеряли: производительность протоколов в production.
Не доказали: отмену любого побочного эффекта после дедлайна.

Что именно распределяет балансировщик

Нужно различать **соединение, запрос, активную работу и стоимость работы**.

Реплика	Запросов	Цена каждого запроса	Суммарная работа
A	100	1 мс CPU	100 мс CPU
B	100	20 мс CPU	2000 мс CPU

Это условный пример: одинаковые счётчики запросов дают разную занятость.

Кроме количества, смотрим **in-flight, очередь, ошибки, latency по репликам**. Readiness помогает решить, куда можно отправлять новую работу.

L4, L7 и долгоживущий канал

Уровень	Единица выбора	Следствие
L4 TCP	Обычно новое соединение	Все байты этого соединения идут выбранной реплике
L7 HTTP/gRPC	Запрос / RPC при поддержке протокола	Можно выбирать backend для каждого нового RPC
Балансировка в gRPC-клиенте	RPC из доступных subchannel	Нужны адреса реплик и подходящая policy

Один канал может нести много RPC. Число вызовов не равно числу соединений.

У gRPC channel может быть несколько subchannel. Длительный stream закреплён за выбранным backend до завершения.

Алгоритмы и неодинаковые запросы

Метод	Когда полезен	Что не знает
Round robin	Похожие реплики и запросы	Текущую стоимость работы
Weighted round robin	Разная известная ёмкость	Мгновенную перегрузку
Least connections	Число соединений отражает занятость	Число RPC внутри HTTP/2-соединения
Least outstanding / latency-aware	Есть подходящие измерения	Будущую стоимость запроса
Hash по ключу	Нужна локальность или affinity	Как устранить горячий ключ

Health check, алгоритм и политика retries решают **разные** задачи.

Демо B. Две реплики и один канал

Прогноз: почему несколько RPC могут попасть в одну реплику?

1. HTTP-запросы через L7: считаем **X-Backend**.
2. Один gRPC-канал через L4: проверяем привязку соединения.
3. gRPC-клиент знает обе реплики: включаем **round_robin**.

```
.venv/bin/python tools/demo.py balance --step grpc-l4
```

Вывод: в одной системе можно балансировать разные единицы работы. Изменение протокола требует проверить путь запроса.

Равные счётчики ещё не равная нагрузка

В стенде: последовательные одинаковые unary-запросы, две здоровые реплики, локальная сеть.

В реальной системе: разная стоимость запросов, потоки, холодные реплики, сбои, очередь, разные зоны.

Чтобы проверить результат, нужны:

- распределение запросов и in-flight по репликам;
- p95/p99 и доля ошибок;
- ресурсы и ограничения общего источника данных.

Добавление реплик приложения не увеличивает ёмкость общей БД автоматически.

Где хранить повторно используемый результат

Место	Что сохраняем	Какую цену платим
Браузер / CDN / reverse proxy	HTTP-представление	Политика freshness, приватность, Vary
Память процесса	Объект или результат вычисления	Отдельные копии на каждой реплике
Общий cache service	Данные по ключу	Сеть, доступность, координация

Cache-aside: чтение кэша; при промахе чтение источника и заполнение кэша.

В стенде кэши находятся **в памяти каждого экземпляра**. Общий источник — учебный сервис хранения. Redis для демонстрации механизма не требуется.

Ключ кэша определяет корректность

```
catalog:v2:tenant-17:book-1:currency-RUB:locale-ru
```

Ключ должен учитывать всё, что меняет допустимый ответ: **tenant, права, валюту, язык, версию схемы**, если они влияют на представление.

Один URL не гарантирует один и тот же ответ для всех пользователей.

> Общий кэш персональных ответов без разделения пользователей может раскрыть чужие данные.

В учебной карточке нет пользователей и персональных цен. Расширение предметной области требует пересмотра ключа.

Попадания, промахи и предел источника

Для одной операции чтения на каждый промах, без retries и фоновых запросов:

$$\lambda_{origin} = \lambda \times (1 - h)$$

Условный сценарий: **10 000 RPS**, hit ratio **95%**, предел источника **2000 RPS**.

Состояние	Нагрузка на источник
Прогретый кэш, h = 0,95	500 RPS
Холодный кэш, h = 0	10 000 RPS

Условие $h > 1 - \mu/\lambda = 0{,}8$ удерживает нагрузку ниже предела в этой модели. Практический запас задаём отдельно. При $h = 0{,}8$ запаса нет.

TTL, инвалидация и гонка заполнения

TTL ограничивает срок жизни записи с момента заполнения. Он не доказывает, что запись отражает свежую версию источника.

Шаг	Читатель	Писатель
1	Прочитал v1, пока не положил в кэш	
2		Записал v2, удалил ключ
3	Положил v1 в уже пустой кэш	

Версионирование, согласованное обновление, повторная проверка и выбранная модель консистентности определяют решение.

Инвалидация одной локальной копии не удаляет копии остальных реплик. Каталог и подтверждение цены имеют разные требования.

Одновременный промах

12 клиентов одновременно не находят популярную карточку.

Режим	Что происходит
Без координации	Несколько одинаковых чтений источника
Lock / singleflight на ключ	Один заполняет, остальные ждут и перечитывают кэш
Две реплики с локальными lock	Заполнение возможно отдельно на каждой реплике

TTL jitter рассеивает одновременное истечение разных ключей. Он не объединяет запросы к одному уже пустому ключу.

Защита от stampede уменьшает работу источника, но ожидание клиентов остаётся.

Демо C. Цена, TTL и очередь за промахом

Прогноз: увидим ли новую цену сразу после изменения источника?

1. MISS → HIT: смотрим число чтений источника.
2. Меняем цену: сравниваем данные источника и кэша.
3. Проверяем инвалидацию, затем ждём TTL.
4. Даём конкурентный холодный старт без lock и с lock.

```
.venv/bin/python tools/demo.py cache --step stampede
```

Доказательство: версии, цены, cache status и origin calls.
Миллисекунды зависят от ноутбука, число лишних чтений объясняет механизм.

Почему среднее скрывает проблему

Условное распределение: **95% hit за 2 мс**, **5% miss за 100 мс**.

$$E[T] = 0{,}95 \times 2 + 0{,}05 \times 100 = 6{,}9 \text{ мс}$$

При этом p99 такого распределения — **100 мс**.
Перцентили не вычисляются взвешиванием перцентилей hit и miss.

На холодном старте меняется и доля промахов, и очередь перед источником.

Проверяем вместе: latency, ошибки, hit ratio, origin RPS и in-flight. Конкретные меры управления перегрузкой продолжим на W7.

Разбор архитектурного решения

Условия: браузер + внутренний сервис, 10 000 чтений/с, источник до 2000/с, цена в каталоге может отставать на 10 секунд. Цена заказа должна быть подтверждена.

Сформулируйте решение:

1. Какие интерфейсы и почему?
2. На каком уровне балансируем? Откуда берутся адреса реплик?
3. Что входит в ключ кэша? Как выполняется требование по свежести?
4. Что проверяем после рестарта кэша?

Разбор: одного TTL = 10 с недостаточно для строгой гарантии свежести. Нужно определить источник версии, гонки заполнения и путь подтверждения цены.

Четыре вопроса перед выбором

Вопрос	Решение, которое надо уметь объяснить
Что обещает контракт?	Представление, ошибки, совместимость, дедлайн
Что распределяем?	Соединения, RPC, активную работу
Какой ответ можно повторить?	Ключ, свежесть, приватность, инвалидация
Чем подтверждаем эффект?	Ошибки, хвосты задержки, origin calls, холодный старт

После занятия: повторить три эксперимента и изменить **одно условие**, заранее записав прогноз.

Проверка понимания

1. Почему 304 не обязательно означает, что сервер не работал?
2. Почему один gRPC-клиент может не нагрузить все реплики?
3. Почему TTL не заменяет модель консистентности?

Для самостоятельной работы: добавить вторую реплику в эксперимент со stampede и объяснить число чтений источника.

На следующем шаге курса: что делать, когда доступной мощности всё равно недостаточно.

История и контракты

- Birrell & Nelson. Implementing Remote Procedure Calls, 1984
- Roy Fielding. REST, глава 5 диссертации, 2000
- Google. Introducing gRPC, 26 февраля 2015
- gRPC Motivation and Design Principles
- gRPC 1.0, 23 августа 2016
- RFC 9110: HTTP Semantics
- Protobuf: Language Guide (proto3)

Транспорт, балансировка и кэширование

- gRPC: Core concepts, Deadlines, Status codes
- gRPC: Custom Load Balancing Policies
- RFC 9113: HTTP/2
- Nginx: HTTP load balancing
- Nginx: stream upstream
- RFC 9111: HTTP Caching
- Nishtala et al. Scaling Memcache at Facebook, NSDI 2013